Model of the Truck ’n Trailer Problem

MPC 2025(Instructor : Francesco Borrelli)

Authors : Eric Chuang (3042017488), Evan Grealish (3041927255), Lennart Peus (3042039731), Seongjae Ahn (3038131605), Toh Wayne (3036062886), Yuwei Chang (3041922432)

Presentation: https://youtu.be/OLZXH1YNP-M?si=W1BDUb455Jln8e8K

Abstract

This report presents the design and implementation of a Model Predictive Control (MPC) framework for the autonomous navigation of an articulated truck-trailer system. Maneuvering such vehicles is challenging due to nonholonomic constraints and the inherent instability of reverse motion, which can induce a jackknife effect. To address this, we adopted a hierarchical development approach, utilizing a kinematic bicycle model to progress from open-loop trajectory validation to a constrained receding horizon scheme. The resulting controller successfully stabilizes the system during non-minimum phase reversing maneuvers and enforces hard constraints to navigate obstacle-dense environments. Simulation results demonstrate that the optimization-based approach effectively generates collision-free trajectories for both forward and backward motions without the need for explicit heuristic rules.

1. Introduction

This paper demonstrates the application of Model Predictive Control (MPC) for the path planning and trajectory tracking of a truck-trailer system. We specifically focus on backward motion, and obstacle avoidance. The core idea is to predict the future behavior of the system and optimize control inputs over a finite horizon, taking into account vehicle dynamics and constraints. The optimization problem is formulated and solved using Pyomo, a Python-based open-source optimization modeling language.

2. Theory and Background

The control of articulated vehicles, particularly in autonomous settings, presents a unique intersection of non-linear dynamics, optimization theory, and constraint management. This section outlines the theoretical frameworks used to model the system and the control strategies to navigate complex environments.

2.1. Kinematics of Articulated Vehicles

To capture the essential behavior of a truck-trailer system without the computational overhead of full dynamic tire modeling, the *Kinematic Bicycle Model* is widely used in the literature. In this framework, the towing vehicle is represented by a steerable front axle and a rigid rear axle, while the trailer is modeled as a link attached via a hitch joint. [1] The defining characteristic of this system is its nonholonomic nature - the vehicle can only move in the direction of its wheels (rolling without slipping). While forward motion is generally stable, the system exhibits critical instability during backward motion due to the *jackknife effect*, where the hitch angle diverges uncontrollably, causing the trailer to collapse against the truck. Consequently, reversing cannot be solved with simple reactive steering but require a predictive approach .[1]

2.2. Model Predictive Control (MPC) Framework

While classical control methods (such as PID) react to current errors, *Model Predictive Control (MPC)* is an optimal control strategy that anticipates future system behavior. Unlike finite-time optimal control, which solves for a trajectory from t_0 to t_{final} once, MPC solves an optimization problem over a finite prediction horizon N at every time step. Only the first control input is applied, and the horizon shifts forward. This allows the system to be "useful for online local motion planning in an uncertain and dynamic environment because it is able to re-plan according to the new available information". [2] The behavior of the controller is defined by minimizing a cost function J, which typically penalizes state deviation (tracking error) and control effort (energy consumption). A distinct advantage of MPC is its ability to explicitly handle system limitations, such as physical steering limits or actuator saturation, ensuring the computed control inputs are physically feasible. [2]

2.3. Environmental Constraints and Safety:

In autonomous navigation, safety is formulated as a constrained optimization problem. The workspace is populated with obstacles that must be avoided, transforming the navigation task into "reaching a goal set, while avoiding collision with static and dynamic obstacles". [2] Mathematically, this is achieved by defining the vehicle and obstacles as geometric sets (e.g., circles or polytopes) and enforcing that their intersection is empty:

$$C_{vehicle} \cap O_{obstacle} = \varnothing \tag{1}$$

Advanced formulations may use *Control Invariant Sets* to guarantee that the system can always remain in a safe state indefinitely, regardless of disturbances. However, even in standard formulations, treating collision avoidance as a hard constraint within the MPC ensures that the solver simply returns a trajectory that results in a crash. [2]

3. Methodology

The development of the truck-trailer autonomous control system followed an iterative, bottom-up design philosophy. Rather than attempting to solve the full non-linear control problem immediately, the project was broken down into distinct stages of increasing complexity. This section outlines the decision-making process, the evolution of the dynamic models, and the progression from simple trajectory optimization to complex, obstacle-aware Model Predictive Control (MPC).

3.1. Model Validation and Forwards Kinematic

In this step, we decided to first test the system dynamics before connecting it to a more complex system prediction model. We also stayed with a simple state equation model. This served as a sanity check for the state-space equations, ensuring that inputs (steering angle, velocity) resulted in physically realistic state updates before any optimization was applied.

3.2. Control Strategy

Once the kinematics were verified, the focus shifted to the control architecture. We adopted a two-step approach to implementing the optimal controller. Initially we formulated the problem as a static optimization task with a finite time constraint. This verified that the solver could find a feasible solution to move the vehicle within a fixed time horizon. The model was then improved by implementing MPC. By solving the optimization problem over a receding horizon, the system gained the ability to react to disturbances and update its trajectory in real-time.

3.3 Reversing

A critical aspect of truck-trailer control is backward motion, which is inherently unstable due to the risk of jackknifing. Going from the previous validation testing, we expanded the simulation to handle both forward and reverse gears). This required modifying the system constraints to allow negative velocities and handling the kinematic singularities often associated with reversing multi-body vehicles. To ensure correct behavior, we stress-tested the controller in "complex scenarios" where the vehicle required precise maneuvering to reach a fixed point backwards. This phase validated that the MPC prediction horizon was sufficiently long to anticipate and prevent jackknifing events.

3.4 Path tracking and Full Autonomy

The final stage of development integrated the previous modules into a cohesive path-tracking system. Instead of moving to a single coordinate, the cost function was updated to minimize deviation from a continuous reference path.The final architecture allows for seamless transition between forward and backward path tracking, while simultaneously maintaining active obstacle avoidance. This represents the culmination of the project, combining the bicycle model dynamics, MPC formulation, and environmental constraints into a unified navigational stack.

4. Model

4.1 System Dynamics

Three different system kinematics were considered. The first model was a geometric model while the second model was found via literature and was a bicycle model. The model consists of two sections; the truck and the trailer. Each section is assumed to be a simple 2D line with a point at either end. Points 1 & 2 are the end points of the truck while points 2 & 3 are the end points of the trailer, (figure 1). Point 1 is the front axle of the truck and point 2 is the rear axle of the truck. The hitch connecting the truck-trailer is assumed to be directly over the rear axle. The trailer also only has one axle, point 3, and is at the back of the trailer.

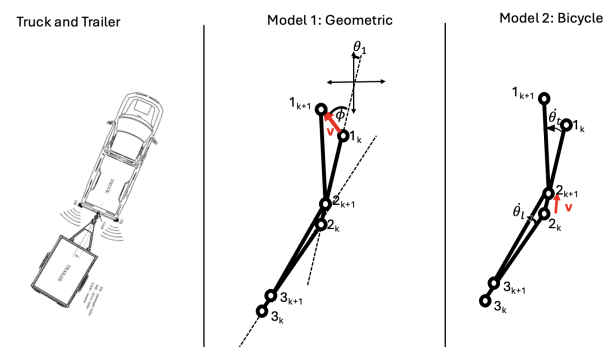

Figure 1: Kinematic Models

4.1.1 Model 1: The Geometric Model

The model assumes a forward wheel drive truck. The inputs were the velocity, v, and steering input, ϕ. These controlled the front wheel axle which was calculated in the next time:

$$\theta_1(k) = atan2(y_2(k) - y_1(k), x_2(k) - x_1(k)) \quad (2)$$

$$x_1(k+1) = x_1(k) + \Delta t \cdot v \cdot sin(\theta_1(k) + \phi(k)) \quad (3)$$

$$y_1(k+1) = y_1(k) + \Delta t \cdot v \cdot cos(\theta_1(k) + \phi(k)) \quad (4)$$

Where $\theta_1(k)$ is the angle of the truck in the global co-ordinates and $x_1(k)$, $x_2(k)$....$y_3(k)$ are the x and y global co-ordinates of the three points.
The middle point 2 in time step k+1 is determined by finding the solution between two geometric equations. The first is that $x_2(k+1)$ & $y_2(k+1)$ will be on the line:

$$A_k x + B_k y + C_k = 0 \tag{5}$$

which passes through $(x_1(k),y_1(k))$ & $(x_2(k),y_2(k))$. The second equation is the distance formula between the first and second point in time step k+1:

$$L_{Truck} = \sqrt{(x_1(k+1) - x_2(k+1))^2, + (y_1(k+1) - y_2(k+1))^2} \tag{6}$$

These equations can be algebraically solved to find ($x_2(k+1)$ & $y_2(k+1)$). The exact same set of operations is then utilized to calculate the co-ordinates of point 3 utilizing points 2_{k+1}.

4.1.2. Model 2: The Bicycle Model

The Bicycle model utilizes a simpler forward Euler method:
$q(k+1) = q(k) + \Delta t \cdot \dot{q}(k)$.
The model uses a set of state variables consisting of x_2, y_2 along with the global angle of the truck-trailer bodies, θ_t and θ_l respectively. It also uses the same two input as Model 1 which are the velocity, v, and steering input, ϕ.
The dynamic derivatives are calculated as follows:

$$\dot{x}_2 = v \cdot cos(\theta_t) \tag{7}$$

$$\dot{y}_2 = v \cdot sin(\theta_t) \tag{8}$$

$$\dot{\theta}_t = \frac{v}{L} \cdot tan(\phi) \tag{9}$$

$$\dot{\theta}_l = \frac{v}{d} \cdot tan(\theta_t - \theta_l) \tag{10}$$

Where L is the truck length and d is the trailer length.
This is a rear wheel drive and forward wheel steering truck The next time step for the four state variables x_2, y_2, θ_t and θ_l are found via the forward Euler method given a time step.

4.1.3. Model 3: The Improved Bicycle Model

Model 3 improved on Model 2 by replacing the inputs with their derivatives. Replacing velocity and steering angle, ϕ, with acceleration and steering rate $\dot{\phi}$ respectively meant a smoother and more realistic velocity and steering angle profile. In model two, point 2 co-ordinates could instantly teleport unrealistically. This replacement with its derivative prevented this phenomenon.
The two new inputs were acceleration, a, and steering rate, $\dot{\phi}$ while there were now 6 state variables, x_2, y_2, θ_t, θ_l, v and ϕ.
There were two additional dynamics equations to go in addition to the original unchanged four dynamic equations.

$$\dot{v} = a \tag{11}$$

$$\dot{\phi}_{State\ Variable\ Derivative} = \dot{\phi}_{Input} \tag{12}$$

4.1.4. Model Selection

Model 2 was chosen over Model 1 as it was more suited to be integrated into an optimal controller and MPC. The chosen Model 2 was then improved to Model 3 to ensure realistic dynamics and this was used in all of the simulations and analysis.

4.2 Model Constraints

Considering our model's state evolves according to a nonlinear kinematic model with state vector

$$x = [x,\ y,\ \theta_t,\ \theta_l,\ v,\ \phi]^T \tag{13}$$

representing planar position, truck-trailer orientations, longitudinal velocity, and steering angle. The discrete-time dynamics used in the optimization problem follow from an Euler discretization of the continuous-time model, encoding the truck-trailer kinematics. The system is subject to state, actuation, geometry, and terminal constraints as described below.
At each prediction step, the discrete-time dynamics constraints guarantee dynamic feasibility of all predicted trajectories. At the same time, physical limits on vehicle actuation impose the following inequalities:

$$a_{min} \leq a_k \leq a_{max} \tag{14}$$

$$\dot{\phi}_{min} \leq \dot{\phi}_k \leq \dot{\phi}_{max} \tag{15}$$

This enforces bounded acceleration and steering-rate behaviour consistent with automotive systems. Similarly, our state constraints ensure feasible steering geometry and safe operations by bounding our velocity and steering angles.
Another implementation for our terminal conditions as its optionality depending on the configuration. We have a hard terminal equality used for exact steering-to-target tasks and no terminal constraints, where convergence is instead enforced through our cost function (equation 18). However, the model cannot be perfectly precise so we chose to set a feasible neighborhood around the target using:

$$\|x_N - x_{target}\|^2 \leq \varepsilon^2 \tag{16}$$

That allows the cost function to reduce to an allowable target range.
When driving backwards, the risk of jackknife becomes apparent so to prevent unsafe articulation between the truck-trailer, we set the relative angle to be constrained to:

$$-\psi_{max} \leq \theta_{t,k} - \theta_{l,k} \leq \psi_{max} \tag{17}$$

Which is often used in truck-trailer literature and serves as a geometric feasibility bound analogous to obstacle avoidance constraints used in optimization-based collision avoidance [1].

4.3 Cost of Trajectory

The cost function has two parts. The first part minimizes the state variables x_2, y_2, θ_t , θ_l by using a squared difference between the current state and final state. The x_2, y_2 dominate until the distance is small.

The second part minimizes the control efforts a and $\dot{\phi}$ via squared terms and a control weighting, W_i.

$$J = \sum_{i=1}^{4} (q_i(N) - t_i)^2 + W_i \cdot \sum_{k\ in\ K} (a_k^2 + \dot{\phi}_k^2) \tag{18}$$

Where J is the objective cost, q is the state variables [x_2, y_2, θ_t , θ_l, v, ϕ] and K is the time set of the horizon of the MPC controller.

4.4 Model Predictive Control Implementation

The overall control objective is to steer the truck-trailer system to a target location while respecting state and input constraints and avoiding jackknife configurations. At each MPC iteration, the optimization problem (equation #) is solved. The cost balances final accuracy with smooth control actions.

Our MPC loop follows a standard receding-horizon structure.

1. Measure current state.
2. Solve the NLP for optimal trajectories.
3. Apply only the first input.
4. Shift horizon and repeat.

To include obstacle prediction and avoidance, enabling safe navigation in environments containing moving objects, the MPC framework incorporates dynamic obstacle prediction and constraint generation directly into the optimization. At each control cycle, the controller predicts the future motion of all detected obstacles over the full prediction horizon of 20 steps (2.0 seconds). A constant-velocity model is used, where the predicted obstacle position evolves as

$$p_o(t) = p_o(o) + v_0 t \tag{19}$$

with position and velocity obtained from onboard sensing. At each prediction step, the controller forms a collision-avoidance constraint between the truck and each obstacle. A safety radius is computed as the sum of the obstacle radius, vehicle radius, and an additional safety margin.

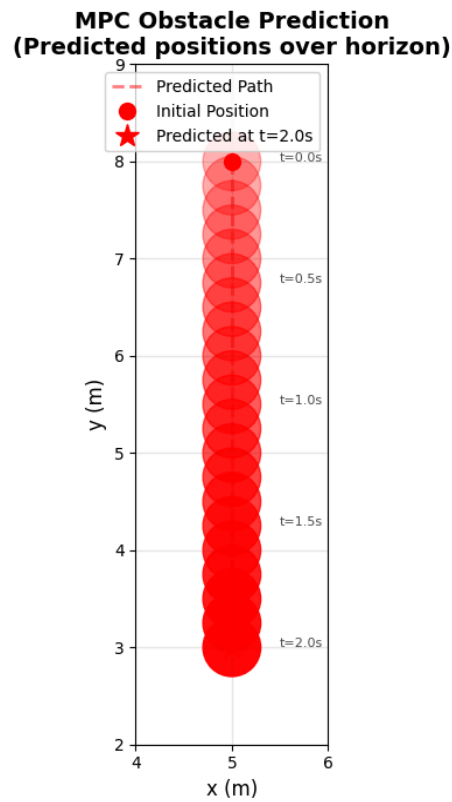

Figure 2: MPC obstacle prediction

The resulting nonlinear constraint is imposed for each obstacle o and horizon time step t:

$$(x(t) - x_o(t))^2 + (y(t) - y_o(t))^2 \geq r_{safe}^2 \tag{20}$$

Thus, for one obstacle and a 21-point horizon, 21 distinct geometric constraints are introduced per MPC iteration. These constraints ensure that the truck-trailer trajectory remains at least 2.5m from all predicted obstacle positions.

After solving the constrained optimization problem, only the first control action is applied. The MPC loop then incorporates updated obstacle measurements, rebuilds the prediction, and resolves the optimization. This repeated replanning makes the controller robust to model error, disturbances, and small deviations in obstacle motion. Predictable obstacle trajectories significantly improve planning quality, while replanning compensates for inaccuracies in the constant-velocity assumption.

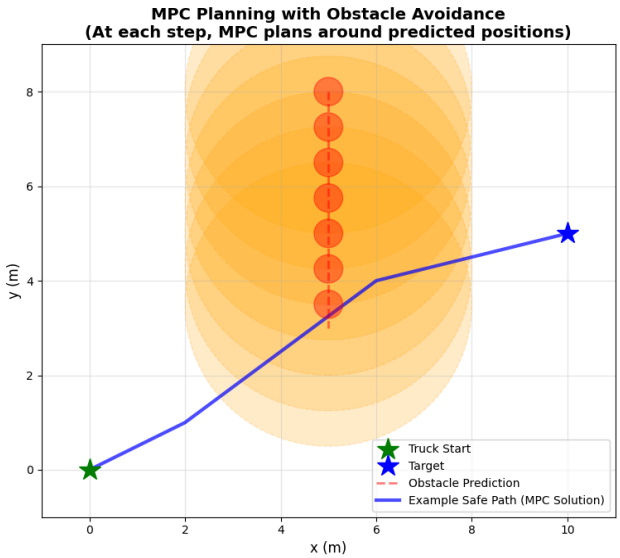

Figure 3: MPC Planning and Obstacle avoidance

4.5 Warm Start Strategy

To ensure real-time performance of the nonlinear MPC framework, a warm-start strategy is incorporated into the optimization routine. Since the system evolves smoothly from one timestep to the next, successive optimal control sequences are highly correlated. Solving each nonlinear program from a zero or arbitrary initial guess would require more iterations, increase computation time, and raise the likelihood of solver divergence, especially given the nonlinearities.

To address this, the optimal control sequence computed at the previous MPC step is used as the initial guess for the next optimization. Specifically, all interior inputs are shifted by one index, and the final input is held constant, preserving the terminal structure implied by the prior solution. A similar shift is applied to the predicted state trajectory to maintain consistency with the truck–trailer dynamics. This "shift-and-hold" warm-start approach leverages the receding-horizon nature of MPC, providing an initialization that remains close to the expected local optimum .

$$u_{0:N-2}^{guess} \leftarrow u_{1:N-1}^{\star} \qquad u_{N-1}^{guess} \leftarrow u_{N-1}^{\star} \tag{21}$$

$$x_{0:N-1}^{guess} \leftarrow x_{1:N-1}^{\star} \qquad x_{N}^{guess} \leftarrow x_{N}^{\star} \tag{22}$$

As a result, IPOPT requires fewer internal iterations to converge, demonstrates more stable behavior across timesteps, and avoids oscillatory or inconsistent solutions. Overall, this warm-start mechanism plays a key role in ensuring the MPC operates reliably at high frequency, enabling the nonlinear controller to produce smooth, dynamically feasible trajectories under articulation constraints. It provides substantial computational advantages with negligible overhead, making it well-suited for real-time implementation of the truck-trailer control system.

5. Simulation and Results

We tested our algorithm on multiple scenarios with randomized moving obstacles. The truck-trailer successfully reversed into the target region while respecting all system constraints. The MPC maintained stable articulation angles throughout the maneuver, with the relative truck–trailer angle consistently remaining inside the jackknife limit. Longitudinal velocity and steering angle remained within allowable bounds, demonstrating that the controller generated dynamically feasible trajectories despite the complexity of reversing with a trailer. Figure 4 shows the applied controls on acceleration and steering angle velocity optimized by our cost function.

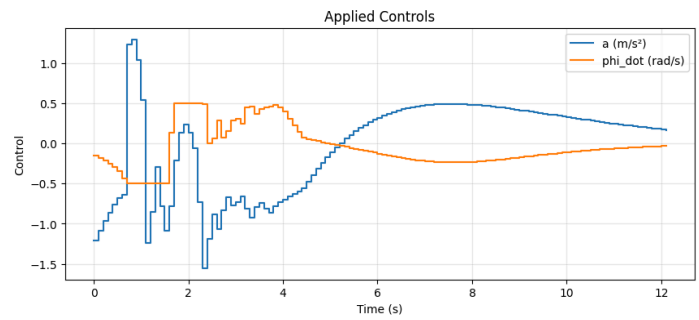

Figure 4: Control parameters when controls are applied

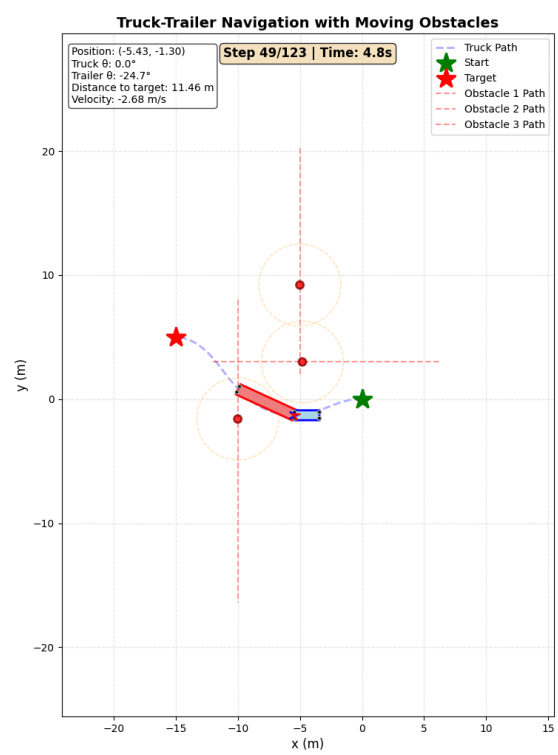

Figure 5: truck-trailer Reverse Navigation with moving obstacles

As seen in figure 5, the results demonstrate that the proposed MPC architecture enables safe, stable, and collision-free backwards driving even under dynamic obstacle motion. The path of our truck-trailer is seen moving from the green star to red star at

time step 4.8s while successfully avoiding the three moving obstacles. The controller balances target tracking with constraint satisfaction, producing smooth and feasible steering commands suitable for real-time implementation.

An alternative approach of MPC with Linearized Dynamics and P–Q–R Cost Matrices was tested but because the system lacks a universal equilibrium point, the dynamics were linearized at each MPC step before solving the CFTOC problem. Q, R & P were made equal to identity matrices since time-varying linearizations prevent the invariant set from being consistently valid. The positive penalties on x and y undesirably encouraged the solver to reduce these coordinates (Figure 6.(a)). Adjusting Q and P to have negative penalties forces the system to move forward but overshoot the target as simulation length increases (Figure 6.(b)). Similarly, in the backward-motion case, identity matrices initially work but still lead to overshooting (Figure 6.(c)).

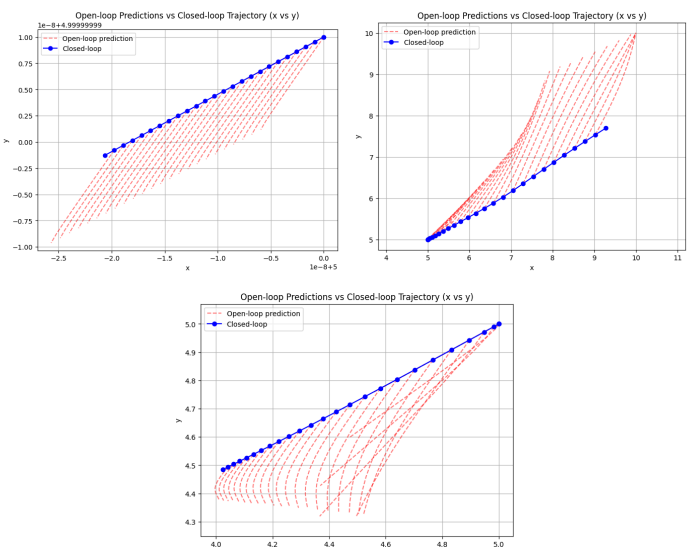

Figure 6(a),(b),(c)

6. Conclusion and Future Improvements

The implementation of Model Predictive Control (MPC) successfully stabilized the truck-trailer system during complex bi-directional maneuvers. A key finding was the controller's ability to handle the *non-minimum phase* dynamics of reversing - where the truck must steer right to turn the trailer left - without explicit logical rules. By optimizing over a finite prediction horizon, the MPC inherently anticipates the onset of the *jackknife effect,* applying corrective steering early to maintain stability.

For the alternative approach, the behavior reveals that this MPC formulation is not naturally suited to our problem. Achieving correct motion requires specific tuning of the P and Q matrices.

The transition to an obstacle-dense environment highlighted the robustness of treating collision avoidance as a hard constraint. The solver consistently generated valid trajectories that satisfied geometric safety margins, effectively guaranteeing that no valid solution would result in a collision. However, the non-convex nature of these constraints occasionally resulted in conservative paths, as the solver avoided local minima associated with tight maneuvering in favor of safer, wider turns.

While the *kinematic bicycle model* proved sufficient for the low-speed docking scenarios tested, it relies on the assumption of "rolling without slipping". This simplifies computation but fails to capture dynamic effects such as tire slip or actuator delay, which are critical at higher velocities. Future development should focus on replacing this with a *dynamic model* that accounts for vehicle mass and inertia to manage lateral slip. Additionally, to bridge the gap between simulation and real-world deployment, future work must address the computational demand of solving nonlinear optimization problems. Implementing linearized solvers (e.g., GRAMPC) or C-code generation would be necessary to ensure deterministic runtimes on embedded hardware.

7. References

[1] Lukassek, M., Dahlmann, J., Völz, A., and Graichen, K., “Model predictive path-following control for truck–trailer systems with specific guidance points: Design and experimental validation,” Mechatronics, vol. 100, Art. no. 103190, 2024.

[2] R. Fallah Firoozi, “Predictive control for motion coordination of connected and automated vehicles,” Ph.D. dissertation, Dept. of Mechanical Engineering, Univ. of California, Berkeley, CA, USA, 2021.